

LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Modul 4: Komunikasi Asinkron dan WebSockets



Disusun Oleh:

Nama : Bella Tri Hapsari

NPM : 25782006

Kelas : 3A

Dosen Pengampu:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra S.T., M.T.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

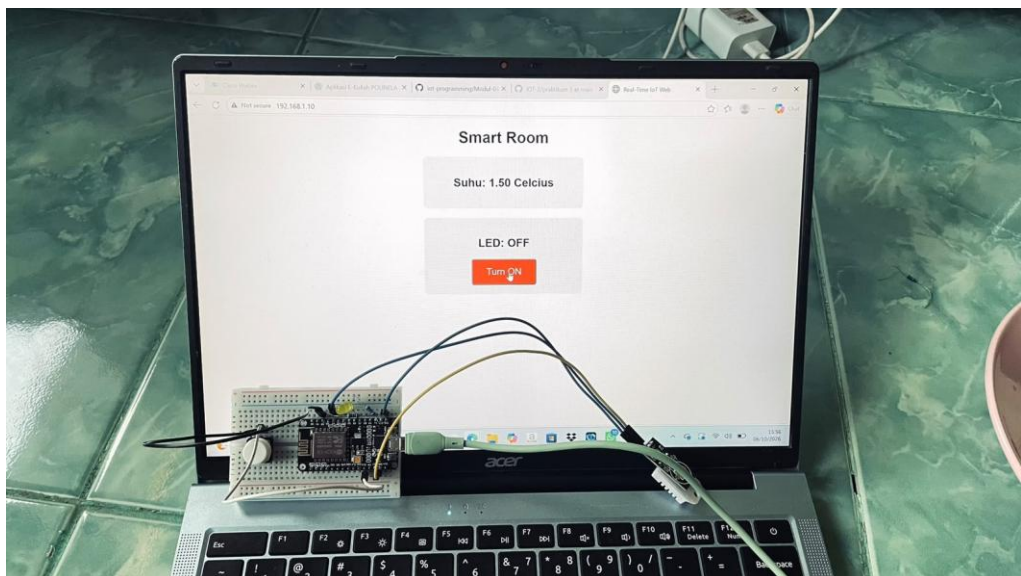
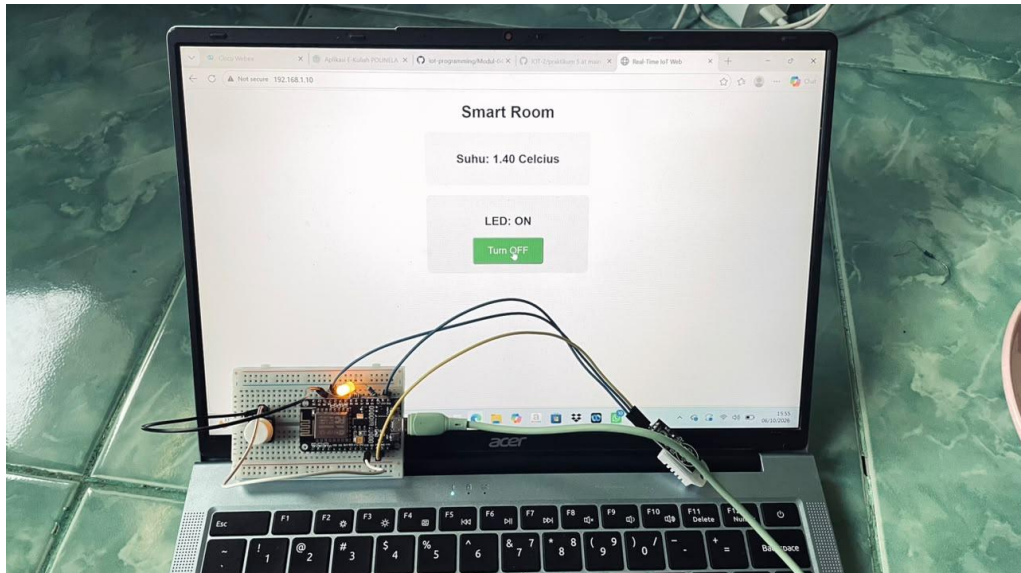
JURUSAN TEKNOLOGI INFORMASI

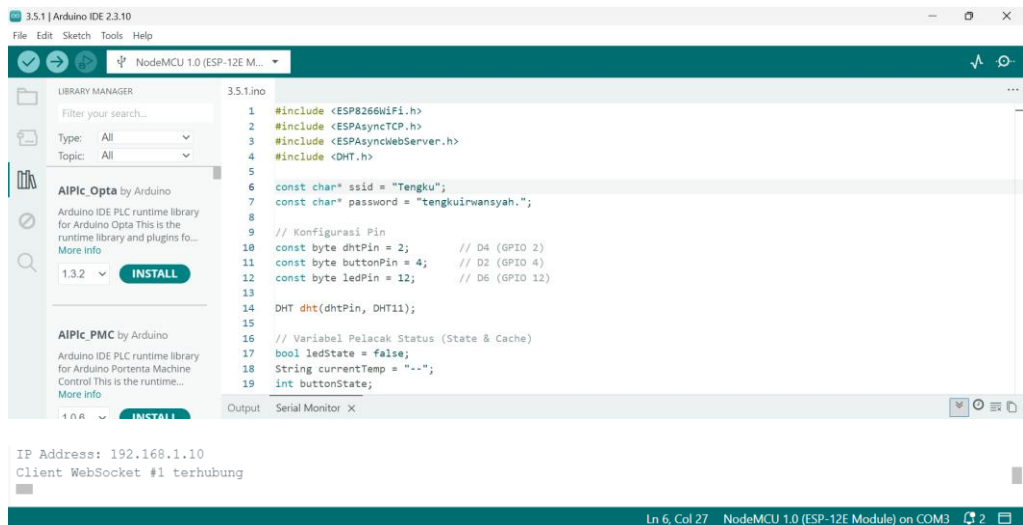
POLITEKNIK NEGERI LAMPUNG

2026/2027

4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*





Penjelasan:

Praktikum Modul 4 bertujuan untuk memahami komunikasi asinkron menggunakan WebSockets pada NodeMCU ESP8266. Pada praktikum ini dibuat sistem yang menghubungkan sensor DHT22, LED, dan push button dengan dashboard web secara real-time. Dengan WebSockets, perubahan status LED dan data suhu dapat dikirim langsung ke browser tanpa melakukan refresh halaman. Praktikum ini juga menggunakan metode non-blocking dengan `millis()` agar NodeMCU tetap dapat merespons input dan komunikasi selama proses pembacaan sensor.

4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab:

WebSockets lebih hemat bandwidth karena koneksi antara browser dan NodeMCU tetap terbuka, sehingga data dapat dikirim langsung ketika ada perubahan tanpa harus meminta dan memuat ulang seluruh halaman web. Pada Meta Refresh HTTP, halaman harus dimuat ulang secara berkala sehingga lebih banyak data yang dikirim dan menyebabkan tampilan halaman berkedip (*flickering*). Dengan WebSockets, hanya data yang berubah saja yang dikirim sehingga tampilan dapat diperbarui secara real-time tanpa refresh.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawab:

Jika menggunakan `delay(3000)`, NodeMCU akan berhenti selama 3 detik setiap kali menjalankan perintah tersebut. Selama waktu itu, proses lain seperti komunikasi WebSocket dan pembacaan tombol tidak dapat merespons dengan cepat. Akibatnya, kontrol LED dari browser maupun tombol fisik dapat mengalami keterlambatan. Karena itu, pada praktikum digunakan `millis()` agar pembacaan sensor tetap berjalan tanpa menghentikan proses lainnya.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawab:

Pada fungsi onMessage(event), terdapat dua elemen HTML yang diperbarui oleh JavaScript, yaitu:

tempValue

ledStatus

tempValue digunakan untuk menampilkan nilai suhu dari DHT11, ledStatus digunakan untuk menampilkan status LED ON atau OFF.

Selain itu, JavaScript juga mengubah tampilan tombol dengan ID:

toggleBtn

Tombol tersebut diubah teks dan kelas CSS-nya sesuai status LED.

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--");. Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
 - Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:

Pada Latihan 4, dilakukan pengembangan sistem sebelumnya dengan menambahkan **pembacaan kelembapan (humidity) menggunakan sensor DHT22** pada dashboard web. Rangkaian yang digunakan tetap sama seperti praktikum sebelumnya, sehingga tidak ada penambahan komponen atau perubahan kabel.

Program membaca **suhu dan kelembapan dari DHT22 setiap 3 detik** menggunakan `millis()` agar proses pembacaan sensor tidak menghentikan proses lain pada ESP8266. Data suhu dan kelembapan kemudian dikirim melalui **WebSocket** ke dashboard secara real-time.

Pada dashboard, ditambahkan bagian **Humidity** sehingga pengguna dapat melihat nilai suhu dalam °C dan kelembapan dalam persen (%). Selain itu, fungsi WebSocket tetap digunakan untuk memperbarui tampilan tanpa perlu melakukan refresh halaman.

Hasil praktikum:

Dashboard berhasil menampilkan **Temperature, Humidity, dan status LED**. Nilai suhu dan kelembapan dari DHT22 dapat diperbarui secara otomatis pada dashboard, sedangkan LED tetap dapat dikontrol melalui tombol pada dashboard maupun push button pada rangkaian.

Bukti praktikum latihan 4

